

Terraform Modules, Refactoring & Terraform Tips and Tricks

WEEK 3

Learning Objectives

- ▶ What are Terraform Modules
- ▶ Module Inputs, Outputs & Locals
- ▶ Writing and Publishing Modules
- ▶ LAB3A: Writing an AWS EC2 Terraform Module
- ▶ Terraform Refactoring
- ▶ Terraform Tips and Tricks
- ▶ Loops & Conditionals
- ▶ Zero-Downtime Deployment

MODULES

HOW TO CREATE REUSABLE INFRASTRUCTURE WITH TERRAFORM MODULES

What is a Terraform Module?

- ▶ In general-purpose languages like Python, you can replace repeated code with a single function to enable reuse, improving efficiency and reducing duplication. (Keeping your code DRY)
- ▶ Terraform modules allow you to package and reuse infrastructure code across multiple environments and projects.
- ▶ A module is a collection of resources that Terraform manages together.
- ▶ A Terraform module is a collection of .tf files used together to provision infrastructure.

What is a Terraform Module?

Benefits

- ▶ Code reuse: define infrastructure components once and reuse them across multiple projects and environments, reducing the need to repeat code.
- ▶ Better organization : Modules act as containers for related resources, breaking down complex infrastructure into smaller, manageable components, which improves readability and organization.
- ▶ Easier maintenance : By modularizing infrastructure, updates or changes can be applied once within a module, cascading across all environments that use it, which reduces the effort needed to maintain code.
- ▶ Standardization : Using a shared set of modules ensures that infrastructure is provisioned consistently across the organization, which reduces configuration drift and improves reliability.
- ▶ Scalability : Terraform allows teams to scale from managing small environments to large, complex architectures by enabling rapid deployment of new infrastructure using existing, tested modules

What is a Terraform Module?

Types of Modules

- ▶ Root Module :
 - ▶ the main working directory where you run Terraform commands like `terraform plan` and `terraform apply`
 - ▶ It is the top-level directory containing your `.tf` files.
 - ▶ It defines the overall infrastructure by calling other modules and configuring providers.
 - ▶ The state file for the entire configuration is usually associated with the root module
- ▶ Child Module :
 - ▶ is any module that is called by another module (often the root module)
 - ▶ Child modules can also call other child modules, creating a tree-like structure

Module Structure

terraform-project/

|

└main.tf

└variables.tf

└outputs.tf

|

└ modules/

└ vpc/

└main.tf

└variables.tf

└outputs.tf

└ README.md

Module Inputs

What are module Inputs

- ▶ To make a function configurable in a general-purpose programming language you can add input parameters to that function
- ▶ In Terraform, modules can have input parameters, too.
- ▶ Inputs serve as the parameters for a module (Variables passed into a module)

Example

- ▶

```
variable "vpc_cidr" {  
  type = string  
}
```


Module Outputs

What are Module Outputs?

- ▶ Terraform module outputs are information exported by a child module to its parent or the user
- ▶ Outputs expose values from a module.
- ▶ They act like **return values** like in a programming function

Example

```
output "vpc_id" {  
  value = aws_vpc.main.id  
}
```

Usage

```
module.vpc.vpc_id
```

Module Locals

What are Locals?

- ▶ Terraform module locals are internal values that allow you to assign a name to an expression, which can then be reused throughout the module
- ▶ They act like **temporary variables** or **constants** that are scoped strictly to the module where they are defined

Example: Defining parameters for a security group

```
locals {  
  http_port = 80  
  any_port = 0  
  any_protocol = "-1"  
  tcp_protocol = "tcp"  
  all_ips = ["0.0.0.0/0"]  
}
```

Writing Modules

- ▶ Define the Purpose of the Module
- ▶ Create the Module Directory Structure
- ▶ Write the Resources in main.tf
- ▶ Define Input Variables in variables.tf
- ▶ Define Outputs in outputs.tf
- ▶ Add Local Values (Optional)

Using a Module

```
module "vpc" {  
  source    = "../modules/vpc"  
  vpc_cidr  = "10.0.0.0/16"  
  vpc_name  = "production-vpc"  
}
```

► Important Parameters

1. source
2. Inputs variables
3. Version (optional)

Module Sources

1. **Local Path** : Use this for modules stored within the same project directory. Custom code under active development.

Format: source = "./modules/vpc"

2. **Git Repository** : Use this to share code across different teams or projects. Private internal libraries.

Format: source = "git::https://github.com/example/vpc-module.git"

3. **Terraform Registry** : Use this for public, community-maintained modules.

Format: source = "terraform-aws-modules/vpc/aws"

Module Versioning

► Why Version Modules?

- Stability
- Predictability
- Safe upgrades

Example

```
► module "vpc" {  
  source = "terraform-aws-modules/vpc/aws"  
  version = "5.0.0"  
}
```

Semantic Versioning

MAJOR.MINOR.PATCH

Example: 2.1.3

Publishing a Module to a Registry

- ▶ Used to share reusable modules.

- ▶ **Steps**

- ▶ Create module
- ▶ Push to GitHub
- ▶ Tag releases
- ▶ Publish to registry

Module Gotchas

Common Mistakes

1. Hardcoding values
2. Missing outputs
3. Circular dependencies
4. Poor naming conventions
5. Not versioning modules

Best Practices

1. Use variables
2. Keep modules small
3. Document modules
4. Use semantic versioning

LAB3A: Writing an AWS EC2 Terraform Module

REFACTORING

REFACTORING

What is Refactoring?

- ▶ Refactoring in Terraform is the systematic process of restructuring your Infrastructure as Code (IaC) to improve its readability, maintainability, and scalability without changing its actual behavior.
- ▶ Improving Terraform code structure without changing infrastructure behavior.
- ▶ Refactoring techniques allow you to update your code structure while ensuring the live infrastructure remains untouched

Reasons for Refactoring

1. Better organization
2. Modularization
3. Easier maintenance
4. Improved readability

REFACTORING

State Manipulation (`terraform state mv`)

- ▶ Move resources within Terraform state.
- ▶ You manually move the resource's identity in the state file
- ▶ **Drawback:** It is imperative and "invisible" to other team members until they run a plan and see the state has already been altered

Example

- ▶ `terraform state mv aws_instance.web module.web.aws_instance.web`

Use Cases

- ▶ Moving resources into modules
- ▶ Renaming resources

REFACTORING

The **moved** Block (Modern Approach)

- It is a declarative statement in your code that tells Terraform a resource's address has changed

Syntax

```
moved {  
  from = aws_instance.old_name  
  to   = aws_instance.new_name  
}
```

REFACTORING

terraform import

- ▶ Import existing infrastructure into Terraform state.

Example

- ▶ `terraform import aws_instance.web i-1234567890`

Benefits

- ▶ Manage existing infrastructure
- ▶ Avoid recreating resources

REFACTORING

Removed Blocks (Terraform 1.7+)

- ▶ Tell Terraform that a resource was intentionally removed.

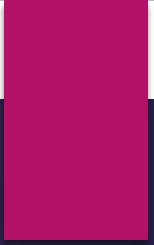
Example

- ▶

```
removed {  
  from = aws_instance.old_server  
}
```

Benefits

- ▶ Cleaner refactoring
- ▶ Safe infrastructure management



TERRAFORM TIPS & TRICKS

LOOPS, IF-STATEMENTS, DEPLOYMENT, AND GOTCHAS

Loops in Terraform

count

- ▶ Terraform's oldest, simplest, and most limited iteration construct:
- ▶ All it does is define how many copies of the resource to create.

Examples

```
resource "aws_iam_user" "example" {  
  count = 3  
  name = "Kojo.${count.index}"  
}  
  
resource "aws_iam_user" "example" {  
  count = length(var.user_names)  
  name = var.user_names[count.index]  
}
```

Loops in Terraform

Loops with for_each Expressions

The for_each expression allows you to loop over lists, sets, and maps to create

- (a) multiple copies of an entire resource,
- (b) multiple copies of an inline block within a resource, or
- (c) multiple copies of a module.

Syntax

```
resource "<PROVIDER>_<TYPE>" "<NAME>" {  
  for_each = <COLLECTION>  
  [CONFIG ...]  
}
```

Example

```
resource "aws_s3_bucket" "bucket" {  
  for_each = toset(["dev", "test", "prod"])  
  bucket = each.key  
}
```

Conditionals in Terraform

Example

- ▶ `instance_type = var.environment == "prod" ? "t3.large" : "t3.micro"`
- ▶ **Benefits**
- ▶ Dynamic configurations
- ▶ Environment-specific deployments

Zero-Downtime Deployment

- ▶ Update infrastructure without service interruption.

Techniques

- ▶ Blue/Green Deployment
- ▶ Rolling Updates
- ▶ Load Balancers
- ▶ Auto Scaling Groups

Terraform Lifecycle Example

- ▶

```
lifecycle {  
  create_before_destroy = true  
}
```

Best Practices Summary

▶ **Recommended Practices**

- ▶ Use modules
- ▶ Version everything
- ▶ Use remote backend
- ▶ Keep code DRY
- ▶ Use variables and outputs
- ▶ Validate changes with terraform plan